- Running time analysis

  - If our partition is balanced (roughly half each side)

partition on n sized array : $\Theta(n)$

$T(n)$ - time for quick sort on array of size 'n'

$$T(n) = 2T(n/2) + \underbrace{\Theta(n)}_{\text{partition}}$$

$\Rightarrow T(n) = n \log n$ (like merge sort)

$\Rightarrow$ optimal

- If it is imbalanced :

  - order after shuffling is in descending order

$x_0 < x_1 < x_2 < \cdots < x_{n-1}$

after 1st partition $\boxed{x_0}\ \boxed{> x_0}$ $= kn$ steps

after 2nd partition $x_0\ \boxed{x_1}\ \boxed{> x_1}$ $= k(n-1)$ steps

$\vdots$

total # partitions $= n-1$

$\Rightarrow \boxed{RT: \Theta(n^2)}$

- comparison based sorting,

  running time $\Omega(n \log n)$

  $\Rightarrow$ we can't do asymptotically better $n \log n$.

- If we shuffle beforehand,

  Average case running time $= O(n \log n)$

  $\downarrow$

  for one input

		10^3	10^6	10^9
Insertion sort	Home (10^8/s)	instant	2.8 hours	31.7 years
	Super (10^{12}/s)	instant	1s	1 week
Merge sort	Home	instant	1s	18 min
	Super	instant	instant	instant
Quick sort	Home	instant	0.6s	12 min
	Super	instant	instant	instant

Good algos are better super computer

Great algos are better than good algos.

- No duplicates:
  - Best: $n \log n$
  - Average: $n \log n$
  - Worst: n^2

$\le p$	p	$> p$

- All elements same & quick sort

$x \quad x \quad x \quad \dots \quad x$

After first partition: [less than or equal to x | x]

After 2nd partition: [$\le x$ | x | x]

⋮

$\Rightarrow RT = \Theta(n^2)$ when all elements are same

$\Downarrow$

also when there are few unique keys (large # duplicates)

- Strategy 1:

$\le p$	p	$\ge p$

- Put the duplicates of 'p' in both left & right buckets to keep it balanced

pivot

3	1	4	3	2	3	6	3	3	4	3	3

partition using original strategy: 3 3 1 3 3 2 3 3 $\boxed{3}$ 4 6 4 (pivot)

partition using strategy 1: 1 3 2 3 3 $\boxed{3}$ 4 3 4 3 6 5 (pivot)

- Strategy 2 → 3 way partition

all duplicates of pivot go into their places

lo … j	j … k	k … hi
$< p$	$= p$	$> p$

- lots of duplicates ⇒ quicksort has to be run on smaller sections recursively

- Best case: when all duplicates

$\Rightarrow RT = \Theta(n)$

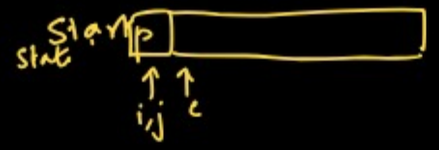
pivot

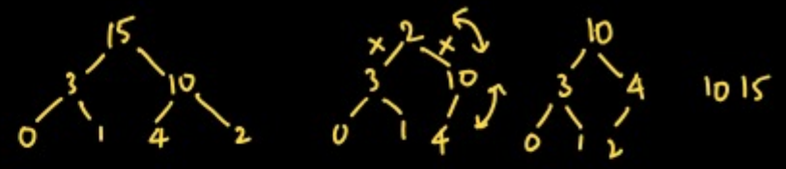
3	1	4	3	2	3	6	3	3	4	3	3

0	1	2	3	4	5	6	7	8	9	10	11
2	1	3	3	3	3	3	3	3	4	6	4

qs(0,1) qs(9,11)

Inplace $\Theta(n)$ 3-way partition



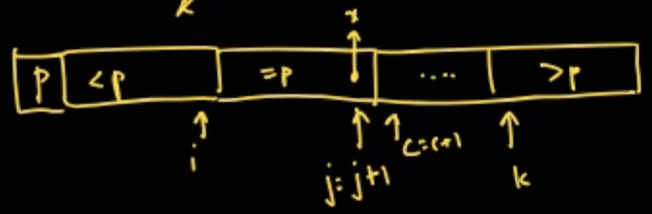


10 15

x goes into =p section

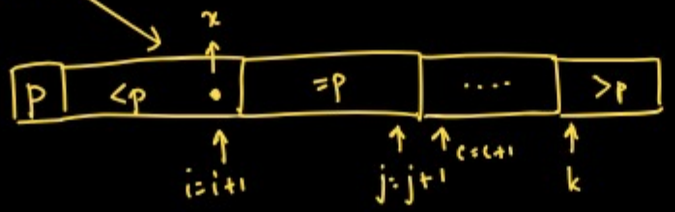
$x==p$

c++
j++



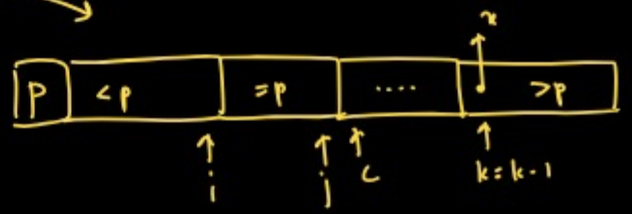
$x<p$

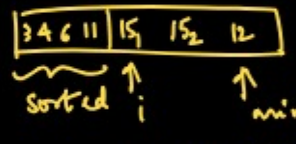
Swap(i+1, c)
i++, j++, c++



$x>p$

Swap(c, k-1)
k--

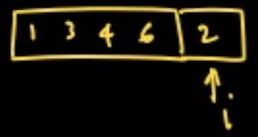




swap(i, min)

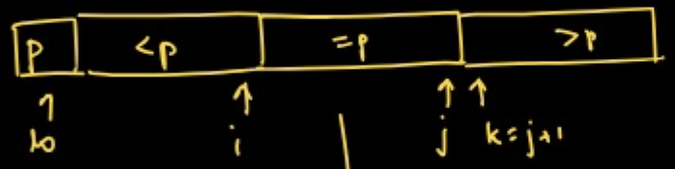
12 15_2 15_3 15_4 15_5 15_1

i



until $j \neq k-1$

final state of invariant



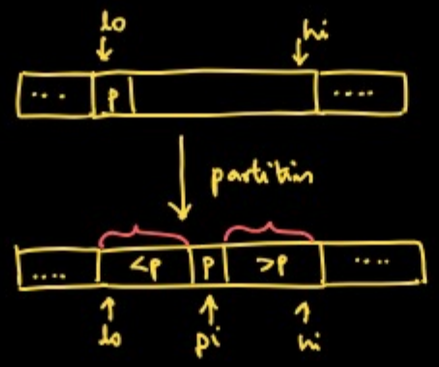
swap(i, lo)

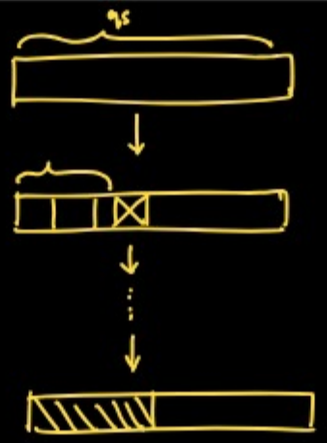
returns i, j (start & end indices of pivot)

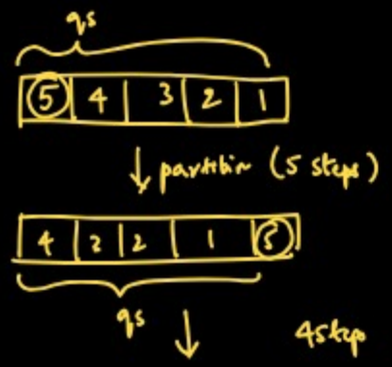
```
quicksort ( arr, lo, hi) {

    pivot_index = partition(arr, lo, hi)
    quicksort(arr, lo, pivot_index-1)
    quicksort(arr, pivot_index+1, hi)

}
```







n

n-1

n-2

⋮

$\Theta(n^2)$

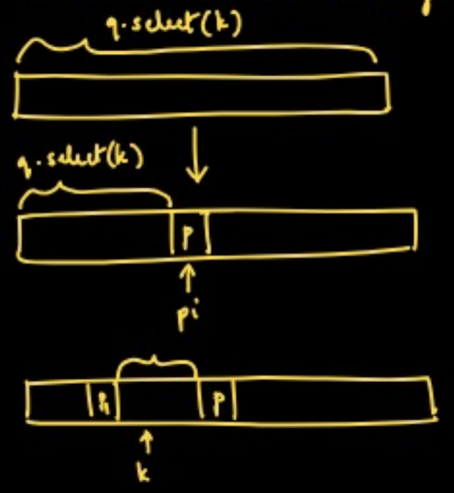
- **Selection problem**
  - Input: array, k
  - Output: k^{th} largest element in the array

$k=0 \Rightarrow$ min
$k=n-1 \Rightarrow$ max
$k=n/2 \Rightarrow$ median

- Easiest solution:
  - sort array
  - return array[k]

$\Big\}\ \Theta(n \log n)$

- Quick select: Based on quick sort & partitioning
  → runs in $O(n)$ time on average.



k^{th} largest

1st partition takes n steps
2nd partition takes $n/2$ steps
3rd partition takes $n/4$ steps
⋮
4
2
1

Total RT (on avg) $= \underbrace{n + n/2 + n/4 + \cdots + 4 + 2 + 1}_{2^0 + 2^1 + 2^2 + \cdots + 2^{\log_2 n}} = 2n-1 = \Theta(n)$

$= \underbrace{1 + 2 + 4 + \cdots + n/2 + n}_{\#\text{terms} = \log_2 n + 1}$

$2+4+8+16+32 = 62$
Sum $=62$
#terms $=5$

$\underbrace{a + ar + ar^2 + ar^3 + \cdots + ar^{m-1}}_{\text{'m' terms}}$

$$\boxed{S_m = \frac{a(r^m-1)}{r-1}}$$

$a=1$
$r=2$
$m = \#\text{terms} = \log_2 n + 1$

$$S = \frac{a(r^m-1)}{r-1}$$
$$= 1\,\frac{(2^{\log_2 n+1}-1)}{2-1}$$
$$= 2^{\log_2 n} \times 2^1 - 1$$
$$= \boxed{2n-1}$$

$a^{m+n} = a^m \cdot a^n$

finding Top k elements (largest)
1. Build heap – $O(n)$
2. Remove max from heap 'k' times $\Rightarrow O(k \log n)$

RT: $O(n + k \log n)$

If $k = \frac{n}{\log n}$, then RT $= O\left(n + \frac{n}{\cancel{\log n}} \times \cancel{\log n}\right)$
$= O(2n)$
$= O(n)$

$n = 1$ billion
$\log_2 n = 30$

$k = \frac{n}{\log_2 n} = \frac{1}{30}$ billion $\sim$ 33 million

We can get top 33 million largest elements among 1 billion in linear time $O(n)$

$n = 1$ trillion $= 10^{12}$
$\log_2 n = 40$

$k = \frac{10^{12}}{40} = 0.025$ trillion
$= 25$ billion elements

$n \xrightarrow{\div 2} n/2 \xrightarrow{\div 2} n/4 \downarrow \cdots \downarrow 2 \xrightarrow{\div 2} 1$

#times we need to divide by 2 to get to $1 = \log_2 n$

$16 \xrightarrow{\div 2} 8 \xrightarrow{\div 2} 4 \xrightarrow{\div 2} 2 \xrightarrow{\div 2} 1$
$1 \xrightarrow{\times 2} 2 \xrightarrow{\times 2} 4 \xrightarrow{\times 2} 8 \xrightarrow{\times 2} 16$

#divisions $=4$
#terms $= 4+1 = 5$
1, 2, 4, 8, 16

$$\underbrace{n + (n-1) + (n-2) + \cdots}_{500 \text{ times}}$$

$$\leq \underbrace{n + n + n + \cdots}_{500 \text{ times}}$$

$\leq 500n$ steps $= O(n)$

Radix Sort

Each sort should be stable

532	240	603	240
603	532	513	513
942	942	532	532
888	513	240	603
678	603	942	648
648	888	648	678
513	678	678	888
240	648	888	942

$\rightarrow$ $\rightarrow$ $\rightarrow$

[5, 3, 0, 4,] target=6

pairs

[(0,5), (1,3), (2,0), (3,4), (4,3)]

sort

[(2,0), (1,3), (4,3), (3,4), (0,5)]

nums {3, 1, 4, 3, 10} target=6

hashmap

3:3

1:1

4:2

[eat, tea, tan, ate, nat, bat]

HashSet {eat, ate, tea, tan, nat}

Outer loop

group = List<String> → eat, tea, ate

① eat

isAnagram(eat, tea) true

② tea → group List<String> → tan, nat

③ tan → tan, nat

④ ate

⑤ bat → bat

-1	-1	-1		
-1	-1	-1		
4	2	4		
5	2	5		
2	2	2	-1	
4	3	4	2	
3	3	3	3	

4bits

$$\underset{3}{1}\ \underset{2}{0}\ \underset{1}{1}\ \underset{0}{0} \sim -6$$

signed int

two's complement

$$-1\cdot 2^3 + 0\cdot 2^2 + 1\cdot 2^1 + 0\cdot 2^0$$

$$-8 + 0 + 2 + 0 = -6$$

min number $1000 = -8$

-2^3

max number $0111 = 2^3 - 1 = +7$

$$2^2 + 2^1 + 2^0 = 2^3 - 1$$

4 bits: $[-2^{4-1}, 2^{4-1}-1]$

32 bit: $[-2^{32-1}, 2^{32-1}-1] = [-2^{31}, 2^{31}-1]$

64 bits: $[-2^{63}, 2^{63}-1]$

32 bits

$$\overset{31\ 30\ 29 \qquad 1\ 0}{0\ \underbrace{1\ 1 \cdots 1}_{31\ 1s}} = 2^{30} + 2^{29} + \cdots + 2^1 + 2^0$$

$$= 2^0 + 2^1 + \cdots + 2^{29} + 2^{30}$$

#terms = 31

$a = 1$

$r = 2$

$$S_n = \frac{a(r^n - 1)}{r-1} = \frac{1(2^{31}-1)}{2-1} = 2^{31} - 1$$

$$\overset{31\ 30\ 29 \qquad 1}{1\ \underbrace{0\ 0 \cdots 0}_{31\ 0s}} = -1\cdot 2^{31} = -2^{31}$$

Unsigned

- ____

min: $0000 = 0$

max: $1111 = 15$

$$2^3 + 2^2 + 2^1 + 2^0 = 2^4 - 1$$

- 32 bits

min: $\underbrace{00 \cdots 0}_{32\ 0s} = 0$

max: $\underbrace{111 \cdots 1}_{32\ 1s} = 2^{32} - 1$

$$2^0 + 2^1 + \cdots + 2^{31} = 2^{32} - 1$$

Each number is $[-8, +7]$ $\Rightarrow$ 4 bits needed to store

①	5 ✓0101	min: 5	0101	0101
②	-3 ✓1101	min: -3	1101	1101
③	4 ✓0100	min: -3	1101	0100
④	3 ✓0011	min: -3	1101	0011
⑤	-4 ✓1100	min: -4	1100	1100

$x = \underbrace{1101}_{\text{min}}\ \underbrace{0011}_{\text{top}}$

top value $= x \,\&\, \overbrace{00001111}^{\boxed{0x0F}}$

min value $= x >> 4$

$\overbrace{\underbrace{1011}_{B}\ \underbrace{0111}_{7}}^{1\text{ byte}}$

$0xB7 = 1011\ 0111$

· Bit mask

$x = \underbrace{0101}_{\text{preserve these bits}}\ \underbrace{1101}_{\text{make these zero}}\ \underbrace{1110}_{\text{preserve these bits}}$

$\&\ \ 1111\ \ 0000\ \ 1111$

$0101\ \ 0000\ \ 1110$ → Bit mask

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

$x \,\&\, x = x$

$x \,\&\, 0 = 0$

$x \,\&\, 1 = x$

· Shifting:

Left shift

$0011 \rightarrow 3$

$<< 1$

$0110 \rightarrow 6$

3×2^1

Right shift

- arithmetic right shift → the new places would be filled by the leading bit
- logical right shift → the new places would be filled by 0

10110111 $>>3$ (arithmetic) $\quad 111\,10110$

10110111 $>>3$ (logical) $\quad 000\,10110$

4 bits - int
8 bits - long

val = 0110 1010 0110

min = 1010

(unsigned long) 1010

00001010

int - 4 bits
long - 8 bits

long l = 1011 0110

min: 1011 = −5, val: 0110 = 6

val = l & 00001111 (0 F)

= l & 0xFL

00001111

$2^3+2^2+2^1+2^0=15$

long

l & 0xF

1111 int

−1

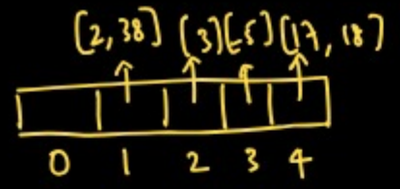
1111 1111 long

−1

[17, 3, −5, 2, 17, 38, 17, 3, 17, −5, −5], k=2

counts { 17:4, 2:1, 38:1, 3:2, −5:3 }

[17, 18]



long packed;

unpacking {
int val = (int) (packed & 0xFL)
int min = (int) (packed >> 4)

packing {
int min, int val
long packed = (((long)min) << 4) | (val & 0xFL)

min = 1011, val = 0110 = 6

1011 0000 | 0000 0110 | 0000 1111

0000 0110

val = 1101 (−3)

val & 0xFL = 0000 1101

1111 1101 (−3) | 0000 1111

min = 1011 (−5)

(long)min = 1111 1011

−8 4 2 1: 1011 = −5

−16 8 4 2 1: 11011 = −5

((long) min) << 4 = 1011 0000

0110 (6)

0000 0110

1011 0000
| 0000 0110
1011 0110

x | 0 = x

x | 1 = 1

x | x = x

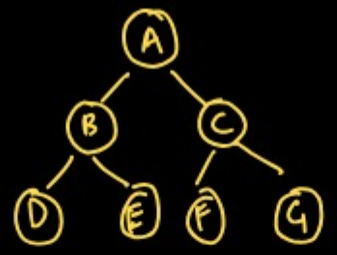
$3+4 = 3\ 4\ +$

Infix notation

$5 \times (3+4) = \overbrace{3\ 4\ +}^{7}\ 5\ \times$

$3+4=7$

$7\times5=35$



inorder: D B E A F C G

postorder: D E B F G C A

Inorder: $(3+4)\times 5$ (infix notation)

Post order: $3\ 4\ +\ 5\ \times$ (postfix notation)

pre order: $\times\ +\ 3\ 4\ 5$ (prefix notation)

5

7 35

3 4 -

4 3 -

4

3

3 4 /

3/4

3 - 4

4

3

```
inorder(node):
    if (node==null) return
    inorder(node.left)
    print(node.value)
    inorder(node.right)
```

```
inorder(A)
    inorder(B)
        inorder(D)
            inorder(null)
            print(D)
            inorder(null)
        print(B)
        inorder(E)
            inorder(null)
            print(E)
            inorder(null)
    print(A)
    inorder(C)
        ⋮
```

D
B
E
A
F
C
G

```
postorder(node)
    //Base case
    postorder(node.left)
    postorder(node.right)
    print(node.val)
```

$A \to B \to C \to D \to E \to F \to G \to H \to I \to$

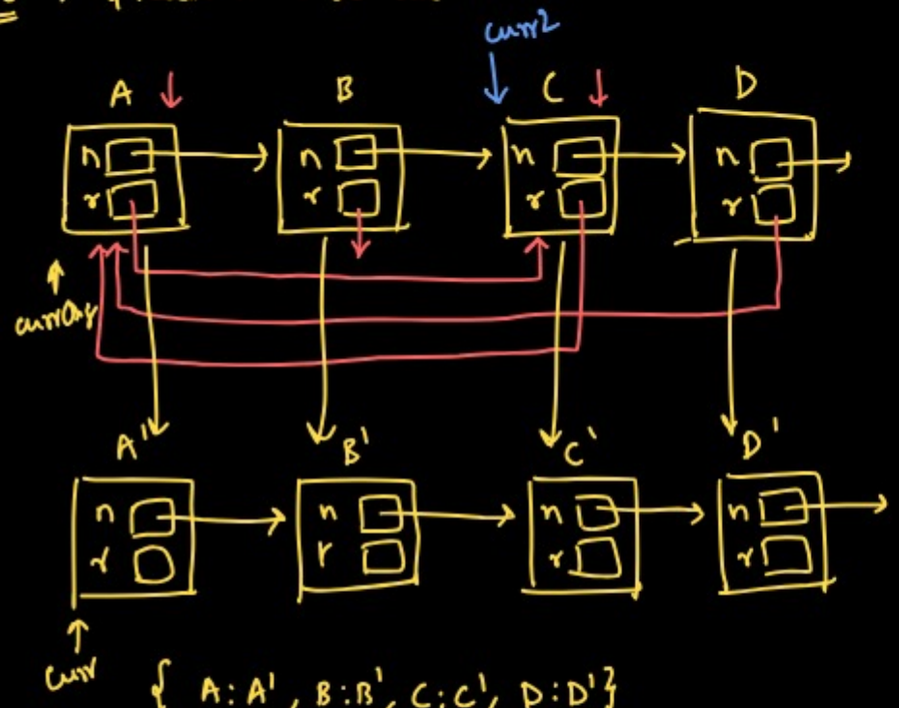
L, r, r, l

$n=3 \Rightarrow$ G needs to be deleted

$A \to B \to C \to D \to$

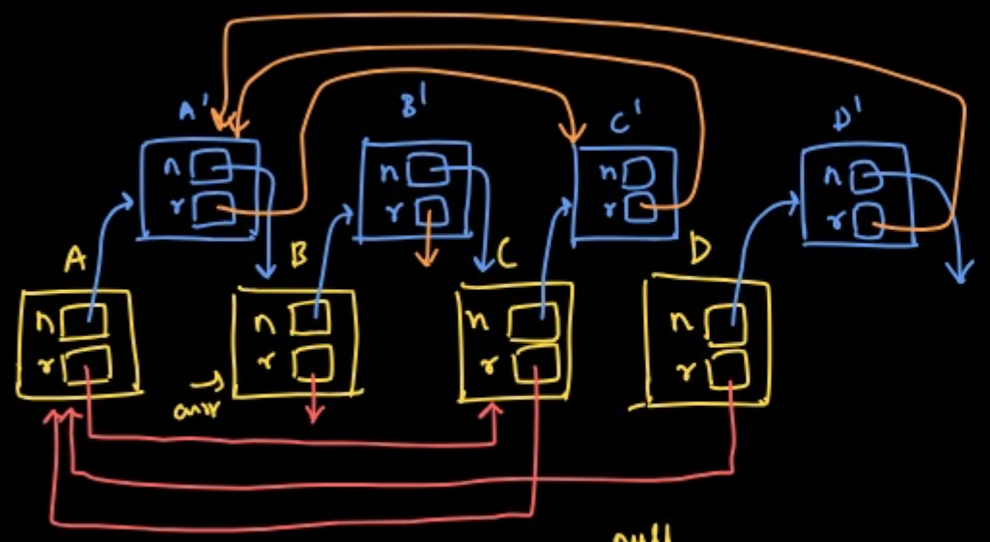
$n=4 \quad n=3 \quad n=2 \quad n=1$

$n=4$



$\{A:A', B:B', C:C', D:D'\}$

curr.random = map.get(currOrig.random);



null

curr.next.random = curr.random.next

A' C

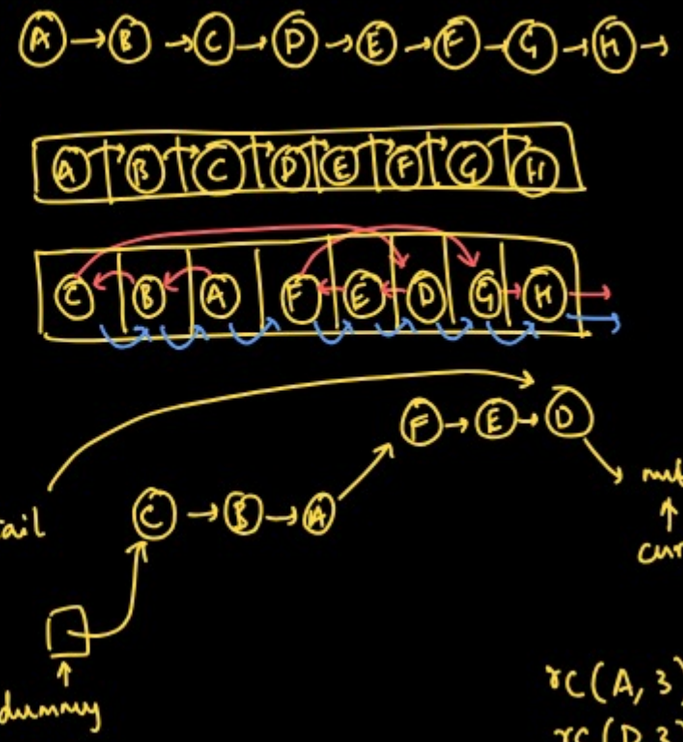
k=3

A → B → C → D → E → F → G → H → I → J → K →

C → B → A → F → E → D → I → H → G → J → K

Strategy2: Reverse each segment without checking if there are k nodes & undo the reversals if there aren't k nodes.

Total work : $\lfloor n/k \rfloor \times k = n$



rc(A, 3) → C

rc(D, 3) → F

rc(G, 3) → G

#segments : $\lfloor n/k \rfloor$

work per segment = 2k ✓ 2 passes ⇒ one for checking if k nodes exist, second pass for the reversal

Total work : $\lfloor n/k \rfloor \times 2k + (n \% k)$ ✗

$= 2n$